



Universidade do Minho
Escola de Engenharia

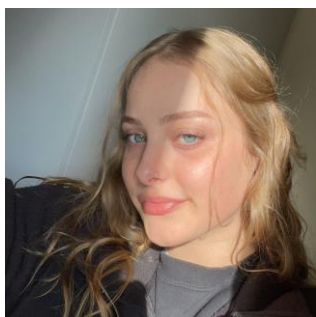
LICENCIATURA EM ENGENHARIA E GESTÃO DE SISTEMAS DE INFORMAÇÃO

Semestre 2 2022/2023

Técnicas de Inteligência Artificial
-TIA-

PROJETO 2

Grupo 6



97042 - Ana Luísa Silva

a97042@alunos.uminho



95601 - André Macedo Barros

a95601@alunos.uminho.pt



91663 - Carlos Rodrigues

a91663@alunos.uminho.pt

Índice

1. Introdução.....	1
1.1 Enquadramento	1
1.2 Objetivos	1
2. Execução do Projeto	2
Projeto 2 (P2) – Itinerário e Meios de Transporte	4
3. Parte A.....	5
3.1 Base de Dados	5
3.2 Sistema de Inferência.....	6
3.3. Interface	7
4. Parte B.....	8
4.1 Base de dados	8
4.2 Sistema de Inferência.....	8
4.3 Interface	9
6. Conclusões	11
6.1 Síntese	11
6.2 Discussão.....	11
6.3 Funcionamento do Trabalho em Grupo	12
Anexos.....	12
Anexo A.....	12
Código Parte A	12
Código Parte B	17
Anexo B.....	28
Contrato do Grupo	28
Bibliografia	29

1. Introdução

1.1 Enquadramento

O Projeto propõe a utilização de um SBC para implementar métodos de Gerar e Testar e Otimização para recomendar vários métodos de transporte para navegar de uma cidade para outra, sendo a condicionante a disponibilidade ou não de métodos de transporte entre as várias cidades e os diferentes custos e tempos entre elas.

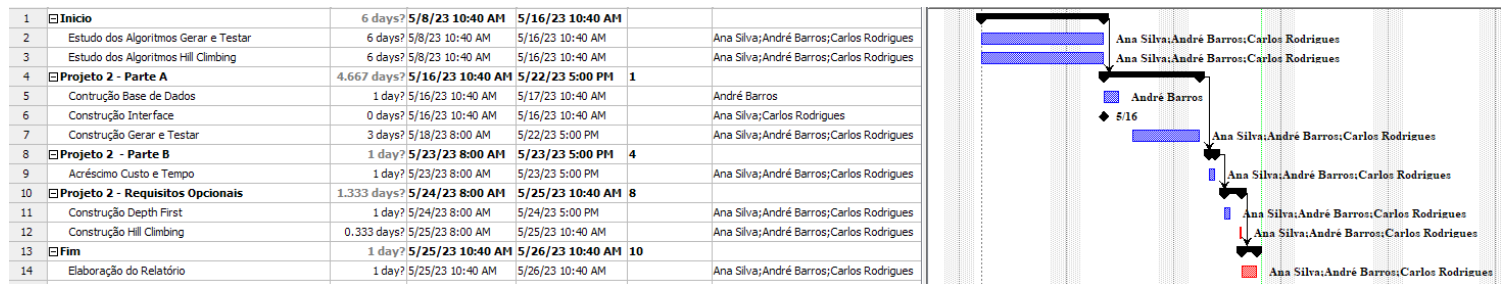
1.2 Objetivos

O objetivo deste projeto é explorar vários algoritmos de procura em grafos, nomeadamente algoritmos segundo o método de Gerar e testar, Depth First e algoritmos de Otimização (Hill Climbing).

Todos estes algoritmos são formas distintas de chegar ao mesmo resultado, no primeiro caso em que se pretende atravessar o grafo o mais economicamente possível, no segundo caso pretende-se atravessar o grafo o mais rapidamente possível e por fim um balanço equilibrado de ambos, distinguindo-se pela forma como chegaram à solução, pelo tempo que demoram a o fazer e pelos recursos computacionais necessários.

2. Execução do Projeto

A calendarização do projeto foi efetuada como na figura seguinte:



Durante a execução deste projeto, adotamos uma abordagem de trabalho em conjunto, reconhecendo a importância da colaboração e da contribuição de cada membro do grupo. Dado o caráter não tão extenso do projeto, dividir as tarefas de forma igualitária não se mostrou uma tarefa fácil. No entanto, encontramos uma solução eficaz: realizamos reuniões regulares onde discutimos ideias, partilhamos conhecimentos e asseguramos que todos os membros pudessem contribuir de forma significativa.

Essas reuniões tornaram-se um espaço para a troca de informações, o esclarecimento de dúvidas e a definição conjunta das próximas etapas do projeto. Valorizamos a diversidade de perspetivas e competências de cada membro da equipa, permitindo que todos dessem a sua contribuição única para o sucesso do projeto.

Ao trabalharmos sempre em conjunto e assegurarmos a contribuição de cada um, maximizamos a nossa eficiência e eficácia. Através de um diálogo aberto, procuramos a compreensão mútua, refinamos as nossas ideias e abordagens, e resolvemos desafios que surgiram ao longo do caminho. Essa sinergia fortaleceu o trabalho em equipa e permitiu-nos alcançar um resultado final coeso e de qualidade.

Reconhecemos que cada membro do grupo possuía competências e conhecimentos valiosos, e através das reuniões, asseguramos que essas contribuições fossem devidamente exploradas e integradas no projeto. Ao fazermos isso, fortalecemos a confiança e o respeito mútuo entre os membros da equipa, o que foi essencial para alcançar um alto nível de colaboração e obter o melhor aproveitamento possível do potencial de cada um.

Posto isto, a contribuição de cada elemento foi a seguinte:

Ana Luísa Silva:

- Elaboração da solução via gerar e testar.
- Elaboração da base de dados.
- Elaboração dos métodos (gerar e testar).
- Aplicação do Sistema de Inferência (Gerar e testar).

Relatório:

- Estruturação do relatório.
- Explicação do funcionamento da parte A e B do gerar e testar.

André Barros:

- Elaboração da solução via gerar e testar
- Elaboração da base de dados
- Análise e pesquisa sobre o problema.
- Elaboração dos métodos (gerar e testar).
- Elaboração da interface para o utilizador.

Relatório:

- Estruturação do relatório.
- Enquadramento.
- Elaboração do contrato de grupo.

Carlos Rodrigues:

Projeto 2 – Parte A:

- Análise e pesquisa sobre o problema.
- Elaboração da base de dados.
- Aplicação de Sistema de Inferência (hill climbing).
- Elaboração da interface para o utilizador.
- Aplicação de Sistema de Inferência do hill climbing e depth first.

Relatório:

- Estruturação do relatório.
- Explicação do funcionamento da parte A e B do hill climbing e depth first.
- Elaboração do diagrama de Gantt.

Projeto 2 (P2) – Itinerário e Meios de Transporte

Neste segundo projeto tivemos de assumir que um viajante tem de fazer um itinerário entre dois locais com meios de transporte intercalares. Na Tabela 1, presente no enunciado, para cada viagem entre dois locais é apresentado o custo de transporte e o tempo despendido. A tabela é simétrica, i.e., se existe viagem de local 1 para local 2 então também é possível viajar de local 2 para local 1 considerando o mesmo Meio de Transporte, Tempo e Custo.

A solução (gerada pelo SBC) foi implementada na linguagem Prolog via uma abordagem de Gerar e Testar e Otimização (por Hill Climbing) e Depth First.

Tabela 1 – Transportes disponíveis

Local 1	Local 2	Meio de Transporte	Tempo (min)	Custo €
Braga	Porto	Comboio	50	3
Braga	Porto	Autocarro	70	14
Braga	Porto	Automóvel	40	15
Braga	Viseu	Automóvel	200	90
Porto	Lisboa	Comboio	180	60
Porto	Lisboa	Automóvel	150	90
Porto	Lisboa	Avião	120	130
Porto	Lisboa	Autocarro	150	25
Lisboa	Faro	Comboio	240	80
Lisboa	Faro	Automóvel	200	150
Lisboa	Faro	Avião	120	140
Lisboa	Faro	Autocarro	240	70
Porto	Viseu	Automóvel	200	90
Porto	Viseu	Comboio	260	70
Porto	Viseu	Autocarro	250	60
Braga	Guimarães	Automóvel	20	10
Braga	Guimarães	Autocarro	30	15
Guimarães	Porto	Automóvel	50	14
Guimarães	Porto	Comboio	70	3
Guimarães	Porto	Autocarro	60	12
Viseu	Coimbra	Automóvel	100	40
Coimbra	Braga	Automóvel	90	60
Coimbra	Lisboa	Comboio	120	30

3. Parte A

Na parte A apresenta-se uma solução implementada em Prolog para encontrar o melhor caminho entre Braga (Local 1) e Lisboa (Local 2), tendo em conta o custo mínimo em transportes. O problema consiste em determinar um itinerário com meios de transporte intercalares, considerando a tabela simétrica de custos e tempos apresentada no enunciado.

Com base na abordagem gerar e testar, o algoritmo percorre todas as possibilidades de caminhos entre a origem e o destino, considerando as viagens disponíveis na tabela de custos e tempos. Para cada estado, o algoritmo calcula o custo total da viagem até aquele ponto. Com base nesses valores, são tomadas decisões para seleccionar o próximo estado a ser explorado, procurando sempre minimizar o custo.

3.1 Base de Dados

GERAR E TESTAR

Neste caso, foram definidos factos que representam as viagens possíveis entre os locais, assim como as informações de custo associadas a cada viagem

A base de dados apresenta informações sobre viagens entre diferentes locais, onde cada viagem é caracterizada pelo meio de transporte utilizado e o custo de viagem. Os locais são representados por cidades, como Braga, Porto, Lisboa, Faro, Viseu, Guimarães e Coimbra.

A relação "arco/3" descreve os arcos entre os locais, especificando as conexões possíveis entre eles. Cada arco está associado a um meio de transporte específico, como comboio, autocarro, avião ou automóvel.

A relação "custo/4" especifica o custo do transporte de um local de origem para um local de destino usando um determinado meio de transporte. Por exemplo, podemos obter o custo de viajar de Braga para Porto de comboio, que é de 3 euros. Da mesma forma, podemos encontrar o custo de viajar de Porto para Lisboa de avião, que é de 130 euros.

Com base nessas informações, é possível realizar consultas e obter respostas sobre os custos de viagem entre diferentes locais utilizando diferentes meios de transporte. Esses dados podem ser úteis para planear os caminhos de viagem, comparar opções de transporte e tomar decisões informadas com base nas preferências pessoais.

DEPTH FIRST & HILL CLIMBING

Para estes dois casos, para além do anterior, foram definidas todas as ligações tal como estão na tabela, resultando assim no predicado ligação/4, sendo que o método de transporte e o destino foram agrupados numa lista por forma a simplificar a regra que imprime a solução na consola.

3.2 Sistema de Inferência

GERAR E TESTAR

O sistema implementa algoritmos para encontrar o caminho de menor custo em um grafo de transporte.

A função caminho/7 é usada para percorrer o caminho de X para Y, calculando o custo total. Ela mantém uma lista de Visitados para evitar loops e armazena o caminho atual em Caminho, o custo acumulado em CustoAtual e os transportes utilizados em Transportes.

A função caminho_menor_custo/5 é usada para encontrar o caminho de menor custo entre dois pontos, X e Y. Ela usa findall/3 para encontrar todos os caminhos possíveis de X para Y e seus respectivos custos. Em seguida, combina os resultados com os caminhos de Y para X e encontra o caminho de menor custo usando a regra menor_custo/4. O caminho resultante é invertido, se necessário, para obter o caminho correto.

A função menor_custo/4 é responsável por encontrar o caminho de menor custo entre uma lista de caminhos e seus respectivos valores. Ela compara o valor atual com o menor valor encontrado até o momento e atualiza a variável correspondente.

As funções inverte_caminho/2 e inverte_transportes/2 são usadas para inverter a ordem dos elementos em uma lista, necessárias quando o caminho resultante precisa ser retornado na ordem correta.

DEPTH FIRST

Semelhante ao gerar e testar, o sistema percorre todos os caminhos possíveis de X para Y, calculando o custo total através da função path/5, de seguida é chamada a função find_min_cost_path/3 que pega na lista de todos os caminhos possíveis e devolve o caminho de menor custo e os respetivos transportes utilizados. Como as ligações inversas estão explícitas na base de dados usada nesta função, não é necessária uma regra para as inverter.

HILL CLIMBING

O hill climb percorre todos os caminhos possíveis de X para Y, mas antes de seleccionar o melhor ele baralha-os de forma aleatória. Como o sistema para de procurar soluções quando deteta que a próxima não melhora, isto é necessário pois reduz a probabilidade da solução devolvida por este sistema não ser a mais ótima possível.

3.3. Interface

O sistema de recomendação de transporte em Prolog apresenta uma interface simples, possuindo três cláusulas principais, 'interface/0', 'origem/0', 'destino/0' e 'algoritmo/0'.

Após a recomendação, o sistema oferece ao utilizador a opção de ser recomendado novamente ou sair do programa.

GERAR E TESTAR

Se o método de resolução escolhido for o de gerar e testar, dependendo da escolha da origem, destino, a função `mais_economica` encontra o caminho mais econômico entre a origem e o destino. Ela exibe o caminho, o custo total e os transportes utilizados. A função `print_caminho` exibe o caminho percorrido e a função `print_transportes` exibe os transportes utilizados.

DEPTH FIRST & HILL CLIMBING

Como a anterior, também exibe o caminho percorrido e o custo total dos transportes, mas utilizando a função `imprimir`.

4. Parte B

Para a parte B, o objetivo era a implementação do custo da viagem, assim, o trabalho foi apenas melhorado com base na parte A.

Assim sendo, na parte B apresenta-se uma solução implementada em Prolog para encontrar o melhor caminho entre Braga (Local 1) e Lisboa (Local 2), tendo em conta o custo mínimo como também o tempo mínimo em transportes.

Na parte B, usando a abordagem de gerar e testar, o algoritmo percorre todas as possibilidades de caminhos entre a origem e o destino, considerando as viagens disponíveis na tabela de custos e tempos. Para cada estado, o algoritmo calcula o custo total e o tempo total da viagem até aquele ponto, assim como a soma entre o produto de cada distância e custo. Com base nesses valores, são tomadas decisões para selecionar o próximo estado a ser explorado, procurando sempre minimizar o custo, o tempo e a soma entre o produto de cada custo e tempo. De forma semelhante, o depth first e o hill climb também tomam esses valores para minimizar o custo.

4.1 Base de dados

Para esta base de dados foi utilizada a base de dados da parte A com o acréscimo de informações do tempo associadas a cada viagem.

GERAR E TESTAR

Neste caso foi implementada a relação "tempo/4" indica o tempo necessário para viajar de um local de origem para um local de destino usando um determinado meio de transporte. Por exemplo, podemos determinar o tempo de viagem de Braga para Porto de automóvel, que é de 40 minutos. Da mesma forma, podemos obter o tempo de viagem de Lisboa para Faro de comboio, que é de 250 minutos.

Com base nessas informações, é possível realizar consultas e obter respostas sobre os custos e tempos de viagem entre diferentes locais utilizando diferentes meios de transporte. Esses dados podem ser úteis para planejar os caminhos de viagem, comparar opções de transporte e tomar decisões informadas com base nas preferências pessoais.

DEPTH FIRST & HILL CLIMBING

A base de dados utilizada é a mesma da parte A para estes dois métodos, sendo que neste aspeto Não há alterações

4.2 Sistema de Inferência

Para a base de conhecimento, foi utilizada a base de conhecimento da parte A, ou seja, para além de implementar algoritmos para encontrar o caminho de menor custo,

também se adicionou para menor tempo e menor multiplicação entre dois pontos em um grafo de transporte.

A função do caminho_menor_tempo/5 encontra o caminho de menor tempo entre dois pontos, X e Y. Ela segue uma lógica similar à regra caminho_menor_custo/5, mas usa o tempo de transporte em vez do custo. A regra caminho_tempo/7 é usada para percorrer o caminho e calcular o tempo total.

A função do caminho_menor_tempo/5 encontra o caminho de menor tempo entre dois pontos, X e Y. Ela segue uma lógica similar à regra caminho_menor_custo/5, mas usa o tempo de transporte em vez do custo. A regra caminho_tempo/7 é usada para percorrer o caminho e calcular o tempo total.

A função do caminho_menor_multiplicacao/5 encontra o caminho de menor multiplicação entre dois pontos, X e Y. Ela segue uma lógica similar às regras anteriores, mas calcula a multiplicação do custo e tempo de transporte para cada caminho. A regra caminho_multiplicacao/7 é usada para percorrer o caminho e calcular a multiplicação total.

Para além do menor_custo/4, as funções menor_tempo/4 e menor_multiplicacao/4 são responsáveis por encontrar o caminho de menor custo, tempo ou multiplicação entre uma lista de caminhos e seus respectivos valores. Elas comparam o valor atual com o menor valor encontrado até o momento e atualizam as variáveis correspondentes.

DEPTH FIRST

Semelhante ao gerar e testar, o sistema percorre todos os caminhos possíveis de X para Y, calculando o tempo total através da função path_b/5, de seguida é chamada a funcao find_min_cost_path_b/3 que pega na lista de todos os caminhos possíveis e devolve o caminho de menor custo e os respetivos transportes utilizados. Como as ligações inversas estão explícitas na base de dados usada nesta função, não é necessária uma regra para as inverter.

HILL CLIMBING

O hill climb percorre todos os caminhos possíveis de X para Y, mas antes de seleccionar o mais rápido ele baralha-os de forma aleatória. Como o sistema pára de procurar soluções quando deteta que a próxima não melhora, isso é necessário pois reduz a probabilidade da solução devolvida por este sistema não ser a mais alta possível.

4.3 Interface

Com base na interface da parte A, foi incrementado mais uma cláusula, possuindo agora quatro cláusulas principais, 'interface/0', 'origem/0', 'destino/0', 'recomendacao/0' e algoritmo/0'. Ou seja, agora será possível opções de recomendação, como o meio de transporte mais económico, o mais rápido ou a combinação de ambos.

GERAR E TESTAR

Se o método de resolução escolhido for o de gerar e testar, as funções auxiliares correspondentes são chamadas: `mais_economica/0`, que encontra o caminho mais econômico entre a origem e o destino, exibindo o caminho, o custo total e os transportes utilizados. A `mais_rapida/0` que encontra o caminho mais rápido entre a origem e o destino, exibindo o caminho, o tempo total e os transportes utilizados, ou a `mais_economica_rapida/0` que encontra o caminho que combina menor custo e menor tempo entre a origem e destino, exibindo o caminho, a soma total de cada multiplicação, e os transportes utilizados.

DEPTH FIRST

Para a utilização do depth first, chama-se a função `custo_tempo_depth`, que funciona de forma muito semelhante a gerar e testar.

HILL CLIMBING

Já o hill climb, chama-se a função `custo_tempo_hill`, que primeiro percorre todas as ligações possíveis e depois com o algoritmo de otimização escolhe a opção mais ótima.

Satisfação dos critérios

Como objetivos opcionais, optou-se por desenvolver uma interface de utilizador que permite ao mesmo escolher o local de chegada, o local de partida, que objetivo pretende e que algoritmo quer que seja usado para obter a solução.

Isto implicou o desenvolvimento da mesma, assim como o desenvolvimento do algoritmo de hill climbing e uma base de dados correspondente, resultando assim na criação dos predicados `ligação/5`, que é uma cópia de cada linha da tabela fornecida.

Requisitos	Descrição	Efetuada
Requisitos Mínimos	A solução (gerada pelo SBC) terá de ser implementada na linguagem Prolog via uma abordagem de Procura via Transição de Estados ou Otimização (por Hill Climbing)	X
Requisitos Opcionais	Definir outras instâncias do problema, para qualquer configuração de partida e chegada	X
	Permitir que o utilizador escolha o método de resolução (e.g., depthfirst, breadthfirst, hill climbing).	X

6. Conclusões

6.1 Síntese

Utiliza-se os métodos de procura e otimização para calcular as rotas entre as cidades tanto para o menor custo como para o menor tempo, através de uma interface fácil e intuitiva de utilizar. A resposta apresentada ao utilizador revela que cidades é necessário atravessar, os meios de transporte a utilizar em cada e ainda o custo total ou tempo total da viagem.

A título opcional foi também implementada funcionalidade para o utilizador escolher as cidades de origem e destino, assim como escolher o algoritmo que pretende utilizar, tendo sido implementado o Gerar e Testar, o Depth First e o Hill Climb.

6.2 Discussão

A solução final atinge todos os objetivos mínimos propostos, e ainda todos os objetivos opcionais propostos, utilizando uma interface de linha de comandos que permite ao utilizador escolher a origem e destino, escolher o objetivo que pretende (mais rápido, mais barato, balanço) e ainda escolher o método que pretende utilizar (gerar e testar, depth first, hill climb), o que torna o programa muito completo. O resultado é impresso na linha de comandos de uma forma formatada e fácil de entender, que delineia o custo da solução, o tempo da solução ou ambos, quando aplicável, e por extenso o caminho que deve tomar, indicando de onde para onde e de que meio de transporte. O desenvolvimento do programa correu como esperado, sendo que a maior dificuldade foi a implementação do algoritmo de hill climbing, onde os resultados produzidos por este não eram os resultados esperados. Visto isto ao fazer uma pesquisa mais aprofundada do algoritmo aprendemos que ele pode ficar preso numa solução local, ou seja, como o algoritmo pára quando não encontra uma melhoria da solução, essa solução apesar de ser melhor que a vizinha, não é necessariamente a solução mais ótima. Para resolver isto, foi implementado um método que “baralha” os vizinhos, por forma a reduzir a probabilidade de encontrar uma solução local. Tendo isto tudo em consideração, consideramos que este projeto cumpre os objetivos estabelecidos.

6.3 Funcionamento do Trabalho em Grupo

Devido a realização conjunta do trabalho em reuniões, o comportamento do grupo foi adequado, em que todos foram responsáveis e contribuíram de igual forma para o projeto e cumpriram o contrato de trabalho e planificação anteriormente referida. Dito isto, não acreditamos que devam existir disparidades de notas entre os elementos do grupo, sugerimos as seguintes avaliações:

Ana:17

André:17

Carlos:17

GRUPO: 17

Anexos

Anexo A

Código Parte A

Base de dados

arco(braga,porto,comboio).
arco(braga,porto,autocarro).
arco(braga,porto,automovel).
arco(braga,viseu,automovel).
arco(porto,lisboa,comboio).
arco(porto,lisboa,automovel).
arco(porto,lisboa,aviao).
arco(porto,lisboa,autocarro).
arco(lisboa,faro,comboio).
arco(lisboa,faro,automovel).
arco(lisboa,faro,aviao).
arco(lisboa,faro,autocarro).
arco(porto,viseu,automovel).
arco(porto,viseu,comboio).
arco(porto,viseu,autocarro).
arco(braga,guimaraes,automovel).
arco(braga,guimaraes,autocarro).
arco(guimaraes,porto,automovel).
arco(guimaraes,porto,comboio).
arco(guimaraes,porto,autocarro).
arco(viseu,coimbra,automovel).
arco(coimbra,braga,automovel).
arco(coimbra,lisboa,comboio).

custo(braga, porto, comboio, 3).
custo(braga, porto, autocarro, 14).
custo(braga, porto, automovel, 15).
custo(braga, viseu, automovel, 90).
custo(porto, lisboa, comboio, 60).
custo(porto, lisboa, automovel, 90).
custo(porto, lisboa, aviao, 130).
custo(porto, lisboa, autocarro, 25).
custo(lisboa, faro, comboio, 80).

```

custo(lisboa, faro, automovel, 150).
custo(lisboa, faro, aviao, 140).
custo(lisboa, faro, autocarro, 70).
custo(porto, viseu, automovel, 90).
custo(porto, viseu, comboio, 70).
custo(porto, viseu, autocarro, 60).
custo(braga, guimaraes, automovel, 10).
custo(braga, guimaraes, autocarro, 15).
custo(guimaraes, porto, automovel, 14).
custo(guimaraes, porto, comboio, 3).
custo(guimaraes, porto, autocarro, 12).
custo(viseu, coimbra, automovel, 40).
custo(coimbra, braga, automovel, 60).
custo(coimbra, lisboa, comboio, 30).

```

```

ligacao(braga, [porto, comboio], 3).
ligacao(braga, [porto, autocarro], 14).
ligacao(braga, [porto, automovel], 15).
ligacao(braga, [viseu, automovel], 90).
ligacao(porto, [lisboa, comboio], 60).
ligacao(porto, [lisboa, automovel], 90).
ligacao(porto, [lisboa, aviao], 130).
ligacao(porto, [lisboa, autocarro], 25).
ligacao(lisboa, [faro, comboio], 80).
ligacao(lisboa, [faro, automovel], 150).
ligacao(lisboa, [faro, aviao], 140).
ligacao(lisboa, [faro, autocarro], 70).
ligacao(porto, [viseu, automovel], 90).
ligacao(porto, [viseu, comboio], 70).
ligacao(porto, [viseu, autocarro], 60).
ligacao(braga, [guimaraes, automovel], 10).
ligacao(braga, [guimaraes, autocarro], 15).
ligacao(guimaraes, [porto, automovel], 14).
ligacao(guimaraes, [porto, comboio], 3).
ligacao(guimaraes, [porto, autocarro], 12).
ligacao(viseu, [coimbra, automovel], 40).
ligacao(coimbra, [braga, automovel], 60).
ligacao(coimbra, [lisboa, comboio], 30).
ligacao(porto, [braga, comboio], 3).
ligacao(porto, [braga, autocarro], 14).
ligacao(porto, [braga, automovel], 15).
ligacao(viseu, [braga, automovel], 90).
ligacao(lisboa, [porto, comboio], 60).
ligacao(lisboa, [porto, automovel], 90).
ligacao(lisboa, [porto, aviao], 130).
ligacao(lisboa, [porto, autocarro], 25).
ligacao(faro, [lisboa, comboio], 80).
ligacao(faro, [lisboa, automovel], 150).
ligacao(faro, [lisboa, aviao], 140).
ligacao(faro, [lisboa, autocarro], 70).
ligacao(viseu, [porto, automovel], 90).
ligacao(viseu, [porto, comboio], 70).
ligacao(viseu, [porto, autocarro], 60).
ligacao(guimaraes, [braga, automovel], 10).
ligacao(guimaraes, [braga, autocarro], 15).
ligacao(porto, [guimaraes, automovel], 14).
ligacao(porto, [guimaraes, comboio], 3).
ligacao(porto, [guimaraes, autocarro], 12).
ligacao(coimbra, [viseu, automovel], 40).
ligacao(braga, [coimbra, automovel], 60).
ligacao(lisboa, [coimbra, comboio], 30).

```

Sistema de inferência

```
%-----gerar e testar-----
```

%encontra o caminho de menor custo entre dois pontos. Ele gera todas as combinacoes de caminhos possiveis e calcula o custo total de cada caminho, selecionando o caminho com o menor custo.

```

caminho_menor_custo(X, Y, Caminho, Custo, Transportes) :-
    % Encontra todos os caminhos possiveis de X para Y e seus custos
    findall([CaminhoAtual, CustoAtual, TransportesAtual], caminho(X, Y, [], CaminhoAtual, 0, CustoAtual, TransportesAtual),
Resultados1),
    % Encontra todos os caminhos possiveis de Y para X e seus custos
    findall([CaminhoAtual, CustoAtual, TransportesAtual], caminho(Y, X, [], CaminhoAtual, 0, CustoAtual, TransportesAtual),
Resultados2),
    % Combina os resultados dos dois sentidos
    append(Resultados1, Resultados2, TodosResultados),
    % Encontra o caminho de menor custo entre todos os resultados
    menor_custo(TodosResultados, CaminhoInvertido, Custo, TransportesInvertidos),
    % Inverte o caminho e a lista de transportes se necessário
    (CaminhoInvertido = [X|_]
-> Caminho = CaminhoInvertido, Transportes = TransportesInvertidos
; inverte_caminho(CaminhoInvertido, Caminho), inverte_transportes(TransportesInvertidos, Transportes)).

% Percorre o caminho de X para Y, calculando o custo total
caminho(Y, Y, Visitados, [Y], CustoAtual, CustoAtual, []) :-
    % Verifica se Y nao foi visitado anteriormente
    \+ member(Y, Visitados).

caminho(X, Y, Visitados, [X|Caminho], CustoAtual, CustoTotal, [Transporte|Transportes]) :-
    % Verifica se X nao foi visitado anteriormente
    \+ member(X, Visitados),
    % Encontra um arco entre X e Z, com o respectivo transporte
    arco(X, Z, Transporte),
    % Verifica se Z nao foi visitado anteriormente
    \+ member(Z, Visitados),
    % Obtem o custo do transporte de X para Z
    custo(X, Z, Transporte, Custo),
    % Calcula o novo custo total
    NovoCusto is CustoAtual + Custo,
    % Percorre recursivamente o caminho de Z para Y
    caminho(Z, Y, [X|Visitados], Caminho, NovoCusto, CustoTotal, Transportes).

% Encontra o caminho de menor custo entre os resultados
menor_custo([[Caminho, Custo, Transportes]], Caminho, Custo, Transportes).

menor_custo([[Caminho, Custo, Transportes]|Resto], CaminhoMenor, CustoMenor, TransportesMenor) :-
    menor_custo(Resto, CaminhoAux, CustoAux, TransportesAux),
    % Compara o custo do caminho atual com o menor custo encontrado até agora
    (Custo < CustoAux -> (CaminhoMenor = Caminho, CustoMenor = Custo, TransportesMenor = Transportes); (CaminhoMenor
= CaminhoAux, CustoMenor = CustoAux, TransportesMenor = TransportesAux)).

% Inverte a ordem dos elementos em uma lista
inverte_caminho([], []).
inverte_caminho([H|T], L) :- inverte_caminho(T, X), concatenar(X, [H], L).

% Inverte a ordem dos elementos em uma lista
inverte_transportes([], []).
inverte_transportes([H|T], L) :- inverte_transportes(T, X), append(X, [H], L).

% Concatena duas listas
concatenar([], L, L).
concatenar([H|T], L, [H|D]) :- concatenar(T, L, D).

%-----Depth First-----
% Caso de saída, quando a cidade atual é o destino, retorna o custo acumulado.
% Função de verificação de caminho: O destino é alcançado
path(City, City, _, [], Cost) :- Cost = 0.

% Recursão: Encontrar o caminho de custo mínimo da cidade atual para o destino
path(City, Destination, Visited, [(City, NextCity, Transport) | Path], Cost) :-
    \+ member(City, Visited), % assegurar que a cidade ainda não foi visitada
    ligacao(City, [NextCity, Transport], StepCost), % consultar a base de dados para encontrar a ligação

```



```

path(NextCity, Destination, [City | Visited], Path, RemainingCost),
Cost is StepCost + RemainingCost.

% Função de entrada
custo_depth(Origin, Destination, AllPaths) :-
    findall((Cost, Path),
        (path(Origin, Destination, [], Path, Cost)),
        AllPaths),
    find_min_cost_path(AllPaths, MinCost, MinPath),
    write('-----'),nl,
    format("Custo total: ~w~n", [MinCost]),nl,
    write("Caminho: "),nl,
    imprimir(MinPath).

% Predicado para encontrar o caminho de custo mínimo de uma lista
find_min_cost_path([(Cost, Path)], Cost, Path).
find_min_cost_path([(Cost, Path) | Rest], MinCost, MinPath) :-
    find_min_cost_path(Rest, RestMinCost, RestMinPath),
    (Cost < RestMinCost ->
        (MinCost = Cost, MinPath = Path);
        (MinCost = RestMinCost, MinPath = RestMinPath)
    ).

% Função para imprimir o resultado
imprimir([]) :- nl.
imprimir([(Origem, Destino, Transporte) | Resto]) :-
    nl, write(' '),write('De '), write(Origem),
    write(' para '), write(Destino),
    write(' utilizando '), write(Transporte), write(' '),nl,nl, write(' '),
    imprimir(Resto).

%-----Stochastic Hill Climbing-----
% Caso de saída, quando a cidade atual é o destino, retorna o custo acumulado.
path_a(City, City, _, [], Cost) :- Cost = 0.

% Recursão: Encontrar o caminho de custo mínimo da cidade atual para o destino
path_a(City, Destination, Visited, [(City, NextCity, Transport) | Path], Cost) :-
    \+ member(City, Visited), % Assegurar que a cidade ainda não foi visitada
    ligacao(City, [NextCity, Transport], StepCost), % Consultar a base de dados para encontrar a ligação
    path_a(NextCity, Destination, [City | Visited], Path, RemainingCost),
    Cost is StepCost + RemainingCost.

% Função de entrada
custo_hill(Origin, Destination, MinPath) :-
    findall((Cost, Path),
        (path_a(Origin, Destination, [], Path, Cost)),
        AllPaths),
    stochastic_hill_climbing_a(AllPaths, BestPath),
    write('-----'), nl,
    write("Custo Total: "), BestPath = (X, MinPath), write(X), write(' Euros'), nl,nl, write("Caminho: "),nl,
    imprimir(MinPath).

% Algoritmo para encontrar o melhor caminho
stochastic_hill_climbing_a(Paths, BestPath) :-
    random_permutation(Paths, ShuffledPaths), % Embaralhar as paths aleatoriamente.
    evaluate_paths(ShuffledPaths, BestPath).

% Avaliar o melhor caminho e retorná-lo
evaluate_paths([Path], Path).
evaluate_paths([Path1, Path2 | Rest], BestPath) :-
    evaluate_paths([Path2 | Rest], CurrentBestPath),
    compare_paths(Path1, CurrentBestPath, BestPath).

% Comparar dois caminhos e retornar o melhor
compare_paths(Path1, Path2, BestPath) :-
    Path1 = (Cost1, _),

```

```
Path2 = (Cost2, _),
(Cost1 < Cost2 -> BestPath = Path1 ; BestPath = Path2).
```

Interface

```
:- dynamic(origem/1).
```

```
:- dynamic(destino/1).
```

```
:- [projeto2A_metodos,projeto2A_bd].
```

```
interface :- nl, nl, write('Bem-vindo ao nosso sistema de recomendação de transporte!'), nl,
             write('Necessita de ajuda para recomendação de meio de transporte mais econômico?'), nl,
             write(' "1." - Sim, ajude-me'), nl,
             write(' "0." - Não, quero sair'), nl,
             read(A),
             (
               (A == 1), nl, origem, nl;
               (A == 0), nl, write('Terminou'), nl, halt;
               write('Opção inválida. Por favor, tente novamente. '), nl, interface
             ).
```

```
origem :- nl, nl, write('Qual o seu local de partida?'), nl,
          write(' "1." - Braga'), nl,
          write(' "2." - Porto'), nl,
          write(' "3." - Faro'), nl,
          write(' "4." - Guimaraes'), nl,
          write(' "5." - Viseu'), nl,
          write(' "6." - Lisboa'), nl,
          write(' "7." - Coimbra'), nl,
          read(B),
          (
            (B == 1), nl, assert(origem(braga)), destino, nl;
            (B == 2), nl, assert(origem(porto)), destino, nl;
            (B == 3), nl, assert(origem(faro)), destino, nl;
            (B == 4), nl, assert(origem(guimaraes)), destino, nl;
            (B == 5), nl, assert(origem(viseu)), destino, nl;
            (B == 6), nl, assert(origem(lisboa)), destino, nl;
            (B == 7), nl, assert(origem(coimbra)), destino, nl;
            write('Introduza uma opção válida, por favor, comece de novo!'), nl, nl, origem
          ).
```

```
destino :- nl, nl, write('Qual o seu destino?'), nl,
          write(' "1." - Braga'), nl,
          write(' "2." - Porto'), nl,
          write(' "3." - Faro'), nl,
          write(' "4." - Guimaraes'), nl,
          write(' "5." - Viseu'), nl,
          write(' "6." - Lisboa'), nl,
          write(' "7." - Coimbra'), nl,
          read(C),
          (
            (C == 1), nl, assert(destino(braga)), algoritmo, nl;
            (C == 2), nl, assert(destino(porto)), algoritmo, nl;
            (C == 3), nl, assert(destino(faro)), algoritmo, nl;
            (C == 4), nl, assert(destino(guimaraes)), algoritmo, nl;
            (C == 5), nl, assert(destino(viseu)), algoritmo, nl;
            (C == 6), nl, assert(destino(lisboa)), algoritmo, nl;
            (C == 7), nl, assert(destino(coimbra)), algoritmo, nl;
            write('Introduza uma opção válida, por favor, tente novamente!'), nl, nl, destino
          ).
```

```
algoritmo :- nl, nl, write('Escolha o método de resolução!'), nl,
            write(' "1." - Gerar e testar'), nl,
            write(' "2." - Hill Climbing com Stoichastic'), nl,
            write(' "3." - Depth First'), nl,
            origem(Origem), destino(Destino),
            read(E),
            (
```

```

E ::= 1 -> nl, mais_economica, nl;
E ::= 2 -> nl, custo_hill(Origem, Destino, MinPath), fim, nl;
E ::= 3 -> nl, custo_depth(Origem, Destino, MinPath), fim, nl;
write('Introduza uma opcao valida, por favor, tente novamente!'), nl, nl, algoritmo
).

```

mais_economica :-

```

% Obtém a origem e o destino da viagem
origem(Origem),
destino(Destino),
% Encontra o caminho mais económico entre a origem e o destino
caminho_menor_custo(Origem, Destino, Caminho, Custo, Transportes),
nl, nl, write('O caminho mais económico de '), write(Origem), write(' para '), write(Destino), write(' é:'), nl,
% Inverte o caminho encontrado para exibição
reverse(Caminho, CaminhoReverso),
write('Caminho: '), print_caminho(CaminhoReverso), nl,
write('Custo total: '), write(Custo), write(' Euros'), nl,
write('Transportes utilizados: '), print_transportes(Transportes), nl.

```

% Exibe o caminho

```
print_caminho([Origem|[]]) :- write(Origem).
```

```
print_caminho([Destino, Origem|Resto]) :- print_caminho([Origem|Resto]), write(' -> '), write(Destino).
```

% Exibe os transportes

```
print_transportes([]).
```

```
print_transportes([Transporte|Resto]) :- write(' > '), write(Transporte), write(' '), print_transportes(Resto), fim.
```

fim :-

```

retractall(origem(_)),
retractall(destino(_)), nl, nl, nl,
write('Obrigado pela sua preferencia! Deseja ser recomendado novamente?'), nl, nl,
write(' "1." - Sim, por favor'), nl,
write(' "0." - Nao. Quero sair '), nl,
read(Z), (
    (1 == Z), nl, origem;
    (0 == Z), nl, write('Terminou'), halt;
    write('Introduza uma opcao valida, por favor tente novamente!'), nl, nl, fim).

```

Código Parte B

Base de dados

```

arco(braga, porto, comboio).
arco(braga, porto, autocarro).
arco(braga, porto, automovel).
arco(braga, viseu, automovel).
arco(porto, lisboa, comboio).
arco(porto, lisboa, automovel).
arco(porto, lisboa, aviao).
arco(porto, lisboa, autocarro).
arco(lisboa, faro, comboio).
arco(lisboa, faro, automovel).
arco(lisboa, faro, aviao).
arco(lisboa, faro, autocarro).
arco(porto, viseu, automovel).
arco(porto, viseu, comboio).
arco(porto, viseu, autocarro).
arco(braga, guimaraes, automovel).
arco(braga, guimaraes, autocarro).
arco(guimaraes, porto, automovel).

```

arco(guimaraes,porto,comboio).
arco(guimaraes,porto,autocarro).
arco(viseu,coimbra,automovel).
arco(coimbra,braga,automovel).
arco(coimbra,lisboa,comboio).

custo(braga, porto, comboio, 3).
custo(braga, porto, autocarro, 14).
custo(braga, porto, automovel, 15).
custo(braga, viseu, automovel, 90).
custo(porto, lisboa, comboio, 60).
custo(porto, lisboa, automovel, 90).
custo(porto, lisboa, aviao, 130).
custo(porto, lisboa, autocarro, 25).
custo(lisboa, faro, comboio, 80).
custo(lisboa, faro, automovel, 150).
custo(lisboa, faro, aviao, 140).
custo(lisboa, faro, autocarro, 70).
custo(porto, viseu, automovel, 90).
custo(porto, viseu, comboio, 70).
custo(porto, viseu, autocarro, 60).
custo(braga, guimaraes, automovel, 10).
custo(braga, guimaraes, autocarro, 15).
custo(guimaraes, porto, automovel, 14).
custo(guimaraes, porto, comboio, 3).
custo(guimaraes, porto, autocarro, 12).
custo(viseu, coimbra, automovel, 40).
custo(coimbra, braga, automovel, 60).
custo(coimbra, lisboa, comboio, 30).

tempo(braga, porto, comboio, 50).
tempo(braga, porto, autocarro, 70).
tempo(braga, porto, automovel, 40).
tempo(braga, viseu, automovel, 200).
tempo(porto, lisboa, comboio, 180).
tempo(porto, lisboa, automovel, 150).
tempo(porto, lisboa, aviao, 120).
tempo(porto, lisboa, autocarro, 150).
tempo(lisboa, faro, comboio, 250).
tempo(lisboa, faro, automovel, 200).
tempo(lisboa, faro, aviao, 120).
tempo(lisboa, faro, autocarro, 240).
tempo(porto, viseu, automovel, 200).
tempo(porto, viseu, comboio, 260).
tempo(porto, viseu, autocarro, 250).
tempo(braga, guimaraes, automovel, 20).
tempo(braga, guimaraes, autocarro, 30).
tempo(guimaraes, porto, automovel, 50).
tempo(guimaraes, porto, comboio, 70).
tempo(guimaraes, porto, autocarro, 60).
tempo(viseu, coimbra, automovel, 100).
tempo(coimbra, braga, automovel, 90).
tempo(coimbra, lisboa, comboio, 120).

ligacao(braga, [porto, comboio], 50, 3).
ligacao(braga, [porto, autocarro], 70, 14).
ligacao(braga, [porto, automovel], 40, 15).
ligacao(braga, [viseu, automovel], 200, 90).
ligacao(porto, [lisboa, comboio], 180, 60).
ligacao(porto, [lisboa, automovel], 150, 90).
ligacao(porto, [lisboa, aviao], 120, 130).
ligacao(porto, [lisboa, autocarro], 150, 25).
ligacao(lisboa, [faro, comboio], 240, 80).
ligacao(lisboa, [faro, automovel], 200, 150).
ligacao(lisboa, [faro, aviao], 120, 140).
ligacao(lisboa, [faro, autocarro], 240, 70).

```

ligacao(porto, [viseu, automovel], 200, 90).
ligacao(porto, [viseu, comboio], 260, 70).
ligacao(porto, [viseu, autocarro], 250, 60).
ligacao(braga, [guimaraes, automovel], 20, 10).
ligacao(braga, [guimaraes, autocarro], 30, 15).
ligacao(guimaraes, [porto, automovel], 50, 14).
ligacao(guimaraes, [porto, comboio], 70, 3).
ligacao(guimaraes, [porto, autocarro], 60, 12).
ligacao(viseu, [coimbra, automovel], 100, 40).
ligacao(coimbra, [braga, automovel], 90, 60).
ligacao(coimbra, [lisboa, comboio], 120, 30).
ligacao(porto, [braga, comboio], 50, 3).
ligacao(porto, [braga, autocarro], 70, 14).
ligacao(porto, [braga, automovel], 40, 15).
ligacao(viseu, [braga, automovel], 200, 90).
ligacao(lisboa, [porto, comboio], 180, 60).
ligacao(lisboa, [porto, automovel], 150, 90).
ligacao(lisboa, [porto, aviao], 120, 130).
ligacao(lisboa, [porto, autocarro], 150, 25).
ligacao(faro, [lisboa, comboio], 240, 80).
ligacao(faro, [lisboa, automovel], 200, 150).
ligacao(faro, [lisboa, aviao], 120, 140).
ligacao(faro, [lisboa, autocarro], 240, 70).
ligacao(viseu, [porto, automovel], 200, 90).
ligacao(viseu, [porto, comboio], 260, 70).
ligacao(viseu, [porto, autocarro], 250, 60).
ligacao(guimaraes, [braga, automovel], 20, 10).
ligacao(guimaraes, [braga, autocarro], 30, 15).
ligacao(porto, [guimaraes, automovel], 50, 14).
ligacao(porto, [guimaraes, comboio], 70, 3).
ligacao(porto, [guimaraes, autocarro], 60, 12).
ligacao(coimbra, [viseu, automovel], 100, 40).
ligacao(braga, [coimbra, automovel], 90, 60).
ligacao(lisboa, [coimbra, comboio], 120, 30).

```

Sistema de inferência

```
%-----gerar e testar-----
```

```
%exploram todas as possibilidades de caminhos entre os pontos de origem e destino, calculando o custo ou tempo total de cada caminho e selecionando o caminho com o menor custo ou tempo.
```

```
% Percorre o caminho de X para Y, calculando o custo total
```

```
caminho(Y, Y, Visitados, [Y], CustoAtual, CustoAtual, []) :-
```

```
    % Verifica se ja foi visitado anteriormente
```

```
    \+ member(Y, Visitados).
```

```
caminho(X, Y, Visitados, [X|Caminho], CustoAtual, CustoTotal, [Transporte|Transportes]) :-
```

```
    % Verifica se X não foi visitado anteriormente
```

```
    \+ member(X, Visitados),
```

```
    % Encontra um arco entre X e Z, com o respectivo transporte
```

```
    arco(X, Z, Transporte),
```

```
    % Verifica se Z não foi visitado anteriormente
```

```
    \+ member(Z, Visitados),
```

```
    % Obtém o custo do transporte de X para Z
```

```
    custo(X, Z, Transporte, Custo),
```

```
    % Calcula o novo custo total
```

```
    NovoCusto is CustoAtual + Custo,
```

```
    % Percorre recursivamente o caminho de Z para Y
```

```
    caminho(Z, Y, [X|Visitados], Caminho, NovoCusto, CustoTotal, Transportes).
```

```
% encontra o caminho de menor custo entre dois pontos.
```

```
caminho_menor_custo(X, Y, Caminho, Custo, Transportes) :-
```

```
    % Encontra todos os caminhos possíveis de X para Y e seus custos
```

```
    findall([CaminhoAtual, CustoAtual, TransportesAtual], caminho(X, Y, [], CaminhoAtual, 0, CustoAtual, TransportesAtual), Resultados1),
```

```

% Encontra todos os caminhos possíveis de Y para X e seus custos
findall([CaminhoAtual, CustoAtual, TransportesAtual], caminho(Y, X, [], CaminhoAtual, 0, CustoAtual, TransportesAtual),
Resultados2),
% Combina os resultados dos dois sentidos
append(Resultados1, Resultados2, TodosResultados),
% Encontra o caminho de menor custo entre todos os resultados
menor_custo(TodosResultados, CaminhoInvertido, Custo, TransportesInvertidos),
% Inverte o caminho e a lista de transportes se necessário
(CaminhoInvertido = [X|_])
-> Caminho = CaminhoInvertido, Transportes = TransportesInvertidos
; inverte_caminho(CaminhoInvertido, Caminho), inverte_transportes(TransportesInvertidos, Transportes)).

% Encontra o caminho de menor custo entre os resultados
menor_custo([[Caminho, Custo, Transportes]], Caminho, Custo, Transportes).

menor_custo([[Caminho, Custo, Transportes]|Resto], CaminhoMenor, CustoMenor, TransportesMenor) :-
    menor_custo(Resto, CaminhoAux, CustoAux, TransportesAux),
    % Compara o custo do caminho atual com o menor custo encontrado até aqui agora
    (Custo < CustoAux -> (CaminhoMenor = Caminho, CustoMenor = Custo, TransportesMenor = Transportes); (CaminhoMenor
= CaminhoAux, CustoMenor = CustoAux, TransportesMenor = TransportesAux)).

% Regras para calcular o caminho com o menor tempo
caminho_menor_tempo(X, Y, Caminho, Tempo, Transportes) :-
    % Encontra todos os caminhos possíveis de X para Y e seus tempos
    findall([CaminhoAtual, TempoAtual, TransportesAtual], caminho_menor_tempo(X, Y, [], CaminhoAtual, 0, TempoAtual,
TransportesAtual), Resultados1),
    % Encontra todos os caminhos possíveis de Y para X e seus tempos
    findall([CaminhoAtual, TempoAtual, TransportesAtual], caminho_menor_tempo(Y, X, [], CaminhoAtual, 0, TempoAtual,
TransportesAtual), Resultados2),
    % Combina os resultados dos dois sentidos
    append(Resultados1, Resultados2, TodosResultados),
    % Encontra o caminho de menor tempo entre todos os resultados
    menor_tempo(TodosResultados, CaminhoInvertido, Tempo, TransportesInvertidos),
    % Inverte o caminho e a lista de transportes se necessário
    (CaminhoInvertido = [X|_])
    -> Caminho = CaminhoInvertido, Transportes = TransportesInvertidos
    ; inverte_caminho(CaminhoInvertido, Caminho), inverte_transportes(TransportesInvertidos, Transportes)).

% Percorre o caminho de X para Y, calculando o tempo total
caminho_tempo(Y, Y, Visitados, [Y], TempoAtual, TempoAtual, []) :-
    % Verifica se Y não foi visitado anteriormente
    \+ member(Y, Visitados).

caminho_tempo(X, Y, Visitados, [X|Caminho], TempoAtual, TempoTotal, [Transporte|Transportes]) :-
    % Verifica se X não foi visitado anteriormente
    \+ member(X, Visitados),
    % Encontra um arco entre X e Z, com o respectivo transporte
    arco(X, Z, Transporte),
    % Verifica se Z não foi visitado anteriormente
    \+ member(Z, Visitados),
    % Obtém o tempo do transporte de X para Z
    tempo(X, Z, Transporte, Tempo),
    % Calcula o novo tempo total
    NovoTempo is TempoAtual + Tempo,
    % Percorre recursivamente o caminho de Z para Y
    caminho_tempo(Z, Y, [X|Visitados], Caminho, NovoTempo, TempoTotal, Transportes).

% Encontra o caminho de menor tempo entre os resultados
menor_tempo([[Caminho, Tempo, Transportes]], Caminho, Tempo, Transportes).

menor_tempo([[Caminho, Tempo, Transportes]|Resto], CaminhoMenor, TempoMenor, TransportesMenor) :-
    menor_tempo(Resto, CaminhoAux, TempoAux, TransportesAux),
    % Compara o tempo do caminho atual com o menor tempo encontrado até aqui agora
    (Tempo < TempoAux -> (CaminhoMenor = Caminho, TempoMenor = Tempo, TransportesMenor = Transportes);
(CaminhoMenor = CaminhoAux, TempoMenor = TempoAux, TransportesMenor = TransportesAux)).

```

```

% Inverte a ordem dos elementos em uma lista
inverte_caminho([], []).
inverte_caminho([H|T], L) :- inverte_caminho(T, X), concatenar(X, [H], L).

% Inverte a ordem dos elementos em uma lista
inverte_transportes([], []).
inverte_transportes([H|T], L) :- inverte_transportes(T, X), append(X, [H], L).

% Concatena duas listas
concatenar([], L, L).
concatenar([H|T], L, [H|D]) :- concatenar(T, L, D).

caminho_menor_multiplicacao(X, Y, Caminho, Multiplicacao, Transportes) :-
    % Encontra todos os caminhos possiveis de X para Y e suas multiplicacoes
    findall([CaminhoAtual, MultiplicacaoAtual, TransportesAtual], caminho_multiplicacao(X, Y, [], CaminhoAtual, 0,
    MultiplicacaoAtual, TransportesAtual), Resultados1),
    % Encontra todos os caminhos possiveis de Y para X e suas multiplicacoes
    findall([CaminhoAtual, MultiplicacaoAtual, TransportesAtual], caminho_multiplicacao(Y, X, [], CaminhoAtual, 0,
    MultiplicacaoAtual, TransportesAtual), Resultados2),
    % Combina os resultados dos dois sentidos
    append(Resultados1, Resultados2, TodosResultados),
    % Encontra o caminho de menor multiplicacao entre todos os resultados
    menor_multiplicacao(TodosResultados, CaminhoInvertido, Multiplicacao, TransportesInvertidos),
    % Inverte o caminho e a lista de transportes se necessario
    (CaminhoInvertido = [X|_]
    -> Caminho = CaminhoInvertido, Transportes = TransportesInvertidos
    ; inverte_caminho(CaminhoInvertido, Caminho), inverte_transportes(TransportesInvertidos, Transportes)).

% Percorre o caminho de X para Y, calculando a multiplicacao total
caminho_multiplicacao(Y, Y, Visitados, [Y], MultiplicacaoAtual, MultiplicacaoAtual, []) :-
    % Verifica se Y nao foi visitado anteriormente
    \+ member(Y, Visitados).

caminho_multiplicacao(X, Y, Visitados, [X|Caminho], MultiplicacaoAtual, MultiplicacaoTotal, [Transporte|Transportes]) :-
    % Verifica se X nao foi visitado anteriormente
    \+ member(X, Visitados),
    % Encontra um arco entre X e Z, com o respectivo transporte
    arco(X, Z, Transporte),
    % Verifica se Z nao foi visitado anteriormente
    \+ member(Z, Visitados),
    % Obtem o custo do transporte de X para Z
    custo(X, Z, Transporte, Custo),
    % Obtem o tempo do transporte de X para Z
    tempo(X, Z, Transporte, Tempo),
    % Calcula a multiplicação entre custo e tempo
    Multiplicacao is MultiplicacaoAtual + (Custo * Tempo),
    % Percorre recursivamente o caminho de Z para Y
    caminho_multiplicacao(Z, Y, [X|Visitados], Caminho, Multiplicacao, MultiplicacaoTotal, Transportes).

% Encontra o caminho de menor multiplicacao entre os resultados
menor_multiplicacao([[Caminho, Multiplicacao, Transportes]], Caminho, Multiplicacao, Transportes).

menor_multiplicacao([[Caminho, Multiplicacao, Transportes]]Resto, CaminhoMenor, MultiplicacaoMenor, TransportesMenor) :-
    menor_multiplicacao(Resto, CaminhoAux, MultiplicacaoAux, TransportesAux),
    % Compara a multiplicacao do caminho atual com a menor multiplicacao encontrada até agora
    (Multiplicacao < MultiplicacaoAux -> (CaminhoMenor = Caminho, MultiplicacaoMenor = Multiplicacao, TransportesMenor =
    Transportes); (CaminhoMenor = CaminhoAux, MultiplicacaoMenor = MultiplicacaoAux, TransportesMenor = TransportesAux)).

%-----Depth First(custo)-----

% Caso de saida, quando a cidade atual e o destino, retorna o custo acumulado.
% Função de verificação de caminho: O destino é alcançado
path(City, City, _, [], Cost) :- Cost = 0.

```

```

% Recursão: Encontrar o caminho de custo mínimo da cidade atual para o destino
path(City, Destination, Visited, [(City, NextCity, Transport) | Path], Cost) :-
    \+ member(City, Visited), % assegurar que a cidade ainda não foi visitada
    ligacao(City, [NextCity, Transport], StepTime, StepCost), % consultar a base de dados para encontrar a ligação
    path(NextCity, Destination, [City | Visited], Path, RemainingCost),
    Cost is StepCost + RemainingCost.

% Função de entrada
custo_depth(Origin, Destination, AllPaths) :-
    findall((Cost, Path),
        (path(Origin, Destination, [], Path, Cost)),
        AllPaths),
    find_min_cost_path(AllPaths, MinCost, MinPath),
    write('-----'),nl,
    format("Custo total: ~w~n", [MinCost]),nl,
    write('Caminho: '),nl,
    imprimir(MinPath).

% Predicado para encontrar o caminho de custo mínimo de uma lista
find_min_cost_path([(Cost, Path)], Cost, Path).
find_min_cost_path([(Cost, Path) | Rest], MinCost, MinPath) :-
    find_min_cost_path(Rest, RestMinCost, RestMinPath),
    (Cost < RestMinCost ->
        (MinCost = Cost, MinPath = Path);
        (MinCost = RestMinCost, MinPath = RestMinPath)
    ).

% Função para imprimir o resultado
imprimir([]) :- nl.
imprimir([(Origem, Destino, Transporte) | Resto]) :-
    nl,write(' '),write('De '), write(Origem),
    write(' para '), write(Destino),
    write(' utilizando '), write(Transporte), write(' '),nl,nl,write(' '),
    imprimir(Resto).

%-----Stoichastic Hill Climbing(custo)-----
% Caso de saída, quando a cidade atual é o destino, retorna o custo acumulado.
path_a(City, City, _, [], Cost) :- Cost = 0.

% Recursão: Encontrar o caminho de custo mínimo da cidade atual para o destino
path_a(City, Destination, Visited, [(City, NextCity, Transport) | Path], Cost) :-
    \+ member(City, Visited), % Assegurar que a cidade ainda não foi visitada
    ligacao(City, [NextCity, Transport], Time, StepCost), % Consultar a base de dados para encontrar a ligação
    path_a(NextCity, Destination, [City | Visited], Path, RemainingCost),
    Cost is StepCost + RemainingCost.

% Função de entrada
custo_hill(Origin, Destination, MinPath) :-
    findall((Cost, Path),
        (path_a(Origin, Destination, [], Path, Cost)),
        AllPaths),
    stochastic_hill_climbing_a(AllPaths, BestPath),
    write('-----'), nl,
    write('Custo Total: '), BestPath = (X, MinPath), write(X), write(' Euros'), nl,nl,write('Caminho: '),nl,
    imprimir(MinPath).

% Algoritmo para encontrar o melhor caminho
stochastic_hill_climbing_a(Paths, BestPath) :-
    random_permutation(Paths, ShuffledPaths), % Embaralhar as paths aleatoriamente.
    evaluate_paths(ShuffledPaths, BestPath).

% Avaliar o melhor caminho e retorná-lo
evaluate_paths([Path], Path).
evaluate_paths([Path1, Path2 | Rest], BestPath) :-
    evaluate_paths([Path2 | Rest], CurrentBestPath),

```



```

compare_paths(Path1, CurrentBestPath, BestPath).

% Comparar dois caminhos e retornar o melhor
compare_paths(Path1, Path2, BestPath) :-
    Path1 = (Cost1, _),
    Path2 = (Cost2, _),
    (Cost1 < Cost2 -> BestPath = Path1 ; BestPath = Path2).

%-----Depth First(tempo)-----

% Caso de saída, quando a cidade atual é o destino, retorna o tempo acumulado.
path_b(City, City, _, [], Tempo) :- Tempo = 0.

% Recursão: Encontrar o caminho de tempo mínimo da cidade atual para o destino
path_b(City, Destination, Visited, [(City, NextCity, Transport) | Path], Tempo) :-
    \+ member(City, Visited), % Assegurar que a cidade ainda não foi visitada
    ligacao(City, [NextCity, Transport], StepTempo, _), % Consultar a base de dados para encontrar a ligação
    path_b(NextCity, Destination, [City | Visited], Path, RemainingTempo),
    Tempo is StepTempo + RemainingTempo.

% Função de entrada
tempo_depth(Origin, Destination, AllPaths) :-
    findall((Tempo, Path),
        (path_b(Origin, Destination, [], Path, Tempo)),
        AllPaths),
    find_min_Tempo_path_b(AllPaths, MinTempo, MinPath),
    write('-----'), nl,
    format("Tempo total: ~w~n", [MinTempo]), nl,
    write("Caminho: "), nl,
    imprimir(MinPath).

% Predicado para encontrar o caminho de tempo mínimo de uma lista
find_min_Tempo_path_b([(Tempo, Path)], Tempo, Path).
find_min_Tempo_path_b([(Tempo, Path) | Rest], MinTempo, MinPath) :-
    find_min_Tempo_path_b(Rest, RestMinTempo, RestMinPath),
    (Tempo < RestMinTempo ->
        (MinTempo = Tempo, MinPath = Path);
        (MinTempo = RestMinTempo, MinPath = RestMinPath)
    ).

%-----Stochastic Hill Climbing(tempo)-----

% Caso de saída, quando a cidade atual é o destino, retorna o tempo acumulado.
path_b2(City, City, _, [], Tempo) :- Tempo = 0.

% Recursão: Encontrar o caminho de tempo mínimo da cidade atual para o destino
path_b2(City, Destination, Visited, [(City, NextCity, Transport) | Path], Tempo) :-
    \+ member(City, Visited), % Assegurar que a cidade ainda não foi visitada
    ligacao(City, [NextCity, Transport], StepTempo, _), % Consultar a base de dados para encontrar a ligação
    path_b2(NextCity, Destination, [City | Visited], Path, RemainingTempo),
    Tempo is StepTempo + RemainingTempo.

% Função de entrada
tempo_hill(Origin, Destination, MinPath) :-
    findall((Tempo, Path),
        (path_b2(Origin, Destination, [], Path, Tempo)),
        AllPaths),
    stochastic_hill_climbing(AllPaths, BestPath),
    write('-----'), nl,
    write('Custo Total: '), BestPath = (X, MinPath), write(X), write(' Minutos'), nl, nl, write('Caminho: '), nl,

    imprimir(MinPath).

% Algoritmo para encontrar o melhor caminho
stochastic_hill_climbing(Paths, BestPath) :-

```

```

    random_permutation(Paths, ShuffledPaths), % Embaralha os caminhos aleatoriamente.
    evaluate_pathb2(ShuffledPaths, BestPath).

% Avalia os caminhos e retorna o melhor
evaluate_pathb2([Path], Path).
evaluate_pathb2([Path1, Path2 | Rest], BestPath) :-
    evaluate_pathb2([Path2 | Rest], CurrentBestPath),
    compare_pathb2(Path1, CurrentBestPath, BestPath).

% Compara dois caminhos e retorna o melhor
compare_pathb2(Path1, Path2, BestPath) :-
    Path1 = (Tempo1, _),
    Path2 = (Tempo2, _),
    (Tempo1 < Tempo2 -> BestPath = Path1; BestPath = Path2).

%-----DEPTH FIRST(tempo e custo)-----

% Base case: when the current city is the destination, return the accumulated cost and time.
% Path verification function: Destination is reached.
pathx(City, City, _, [], Cost, Time) :- Cost = 0, Time = 0.

% Recursion: Find the minimum cost and time path from the current city to the destination.
pathx(City, Destination, Visited, [(City, NextCity, Transport) | Path], Cost, Time) :-
    \+ member(City, Visited), % Ensure that the city has not been visited yet.
    ligacao(City, [NextCity, Transport], StepTime, StepCost), % Query the database to find the connection.
    pathx(NextCity, Destination, [City | Visited], Path, RemainingCost, RemainingTime),
    Cost is StepCost + RemainingCost,
    Time is StepTime + RemainingTime.

% Entry function.
custo_tempo_depth(Origin, Destination, AllPaths) :-
    findall((Cost, Time, Path),
        (pathx(Origin, Destination, [], Path, Cost, Time)),
        AllPaths),
    find_min_cost_time_pathx(AllPaths, MinCost, MinTime, MinPath),
    write('-----'), nl,
    format("Custo: ~w Euros~n", [MinCost]), nl,
    format("Tempo: ~w Minutos~n", [MinTime]), nl,
    write('Caminho: '), nl,
    imprimir(MinPath).

% Predicate to find the path with the minimum cost and time from a list.
find_min_cost_time_pathx([(Cost, Time, Path)], Cost, Time, Path).
find_min_cost_time_pathx([(Cost, Time, Path) | Rest], MinCost, MinTime, MinPath) :-
    find_min_cost_time_pathx(Rest, RestMinCost, RestMinTime, RestMinPath),
    (Cost * Time < RestMinCost * RestMinTime ->
        (MinCost = Cost, MinTime = Time, MinPath = Path);
        (MinCost = RestMinCost, MinTime = RestMinTime, MinPath = RestMinPath)
    ).

%-----HILL CLIMB(tempo e custo)-----

% Base case: when the current city is the destination, return the accumulated cost and time.
pathy(City, City, _, [], Cost, Time) :-
    Cost = 0,
    Time = 0.

% Recursion: Find the minimum cost and time path from the current city to the destination.
pathy(City, Destination, Visited, [(City, NextCity, Transport) | Path], Cost, Time) :-
    \+ member(City, Visited), % Ensure that the city has not been visited yet.
    ligacao(City, [NextCity, Transport], TimeStep, CostStep), % Query the database to find the connection.
    pathy(NextCity, Destination, [City | Visited], Path, RemainingCost, RemainingTime),
    Cost is CostStep + RemainingCost,
    Time is TimeStep + RemainingTime.

% Entry function
custo_tempo_hill(Origin, Destination, MinPath) :-

```

```

findall((Cost, Time, Path),
    (pathy(Origin, Destination, [], Path, Cost, Time)),
    AllPaths),
stochastic_hill_climbing_y(AllPaths, BestPath),
write('-----'), nl,
write('Custo: '), BestPath = (X, _, _), write(X), write(' Euros'), nl,nl,
write('Tempo: '), BestPath = (_, Y, _), write(Y), write(' Minutos'), nl,nl,
write('Caminho: '), BestPath = (_, _, Z),nl,
imprimir(Z).

% Algorithm to find the best path considering cost multiplied by time
stochastic_hill_climbing_y(Paths, BestPath) :-
    random_permutation(Paths, ShuffledPaths), % Shuffle the paths randomly.
    evaluate_pathsy(ShuffledPaths, BestPath).

% Evaluate the best path and return it
evaluate_pathsy([Path], Path).
evaluate_pathsy([Path1, Path2 | Rest], BestPath) :-
    evaluate_pathsy([Path2 | Rest], CurrentBestPath),
    compare_pathsy(Path1, CurrentBestPath, BestPath).

% Compare two paths and return the best one based on cost multiplied by time
compare_pathsy(Path1, Path2, BestPath) :-
    Path1 = (Cost1, Time1, _),
    Path2 = (Cost2, Time2, _),
    (Cost1 * Time1 < Cost2 * Time2 -> BestPath = Path1 ; BestPath = Path2).

Interface
:- dynamic(origem/1).
:- dynamic(destino/1).
:- [projeto2B_metodos,projeto2B_bd].

interface :- nl, nl, write('Bem-vindo ao nosso sistema de recomendação de transporte!'), nl,
    write('Necessita de ajuda para recomendação de meio de transporte ?'), nl,
    write(' "1." - Sim, ajude-me'), nl,
    write(' "0." - Não, quero sair'), nl,
    read(A),
    (
        (A == 1), nl, origem, nl;
        (A == 0), nl, write('Terminou'), nl, halt;
        (otherwise), nl,write('Opção inválida. Por favor, tente novamente.'), nl, interface
    ).

origem :- nl, nl, write('Qual o seu local de partida?'), nl,
    write(' "1." - Braga'), nl,
    write(' "2." - Porto'), nl,
    write(' "3." - Faro'), nl,
    write(' "4." - Guimaraes'), nl,
    write(' "5." - Viseu'), nl,
    write(' "6." - Lisboa'), nl,
    write(' "7." - Coimbra'), nl,
    read(B),
    (
        (B == 1), nl, assert(origem(braga)), destino, nl;
        (B == 2), nl, assert(origem(porto)), destino, nl;
        (B == 3), nl, assert(origem(faro)), destino, nl;
        (B == 4), nl, assert(origem(guimaraes)), destino, nl;
        (B == 5), nl, assert(origem(viseu)), destino, nl;
        (B == 6), nl, assert(origem(lisboa)), destino, nl;
        (B == 7), nl, assert(origem(coimbra)), destino, nl;
        write('Introduza uma opção válida, por favor, comece de novo!'), nl, nl, origem
    ).

destino :- nl, nl, write('Qual o seu destino?'), nl,
    write(' "1." - Braga'), nl,
    write(' "2." - Porto'), nl,

```

```

write(' "3." - Faro'), nl,
write(' "4." - Guimaraes'), nl,
write(' "5." - Viseu'), nl,
write(' "6." - Lisboa'), nl,
write(' "7." - Coimbra'), nl,
read(C),
(
(C == 1), nl, assert(destino(braga)),recomendacao, nl;
(C == 2), nl, assert(destino(porto)),recomendacao, nl;
(C == 3), nl, assert(destino(faro)),recomendacao, nl;
(C == 4), nl, assert(destino(guimaraes)),recomendacao, nl;
(C == 5), nl, assert(destino(viseu)),recomendacao, nl;
(C == 6), nl, assert(destino(lisboa)),recomendacao, nl;
(C == 7), nl, assert(destino(coimbra)),recomendacao, nl;
write('Introduza uma opção válida, por favor, tente novamente!'), nl, nl, destino
).

```

```

recomendacao :- write('Necessita de ajuda para recomendação de meio de transporte ?'), nl,
write(' "1." - mais económico'), nl,
write(' "2." - mais rápido'), nl,
write(' "3." - mais económico e rápido'),nl,
read(A),
(
(A == 1), nl,mais_barato , nl;
(A == 2), nl,mais_rapido, nl;
(A == 3), nl,balanco, nl;

write('Opcao invalida. Por favor, tente novamente. '), nl, recomendacao
).

```

%-----

```

balanco :- nl, nl, write('Escolha o método de resolução!'), nl,
write(' "1." - Gerar e testar'), nl,
write(' "2." - Hill Climbing com Stoichastic'), nl,
write(' "3." - Depth First'), nl,
origem(Origem), destino(Destino),
read(E),
(
E == 1 -> nl, mais_economica_rapida, nl;
E == 2 -> nl, custo_tempo_hill(Origem, Destino, MinPath);
E == 3 -> nl, custo_tempo_depth(Origem, Destino, MinPath);
write('Introduza uma opcao valida, por favor, tente novamente!'), nl, nl, balanco
),
fim.

```

```

mais_barato :- nl, nl, write('Escolha o método de resolução!'), nl,
write(' "1." - Gerar e testar'), nl,
write(' "2." - Hill Climbing com Stoichastic'), nl,
write(' "3." - Depth First'), nl,
origem(Origem), destino(Destino),
read(E),
(
E == 1 -> nl, mais_economica, nl;
E == 2 -> nl, custo_hill(Origem, Destino, MinPath);
E == 3 -> nl, custo_depth(Origem, Destino, MinPath);
write('Introduza uma opcao valida, por favor, tente novamente!'), nl, nl, mais_barato
),
fim.

```

```

mais_rapido :- nl, nl, write('Escolha o método de resolução!'), nl,
write(' "1." - Gerar e testar'), nl,
write(' "2." - Hill Climbing com Stoichastic'), nl,
write(' "3." - Depth First'), nl,
origem(Origem), destino(Destino),
read(F),

```

```

(
    F ::= 1 -> nl, mais_rapida, nl;
    F ::= 2 -> nl, tempo_hill(Origem, Destino, MinPath), nl;
    F ::= 3 -> nl, tempo_depth(Origem, Destino, MinPath), nl;
    write('Introduza uma opcao valida, por favor, tente novamente!'), nl, nl, mais_rapido
),
fim.

%-----

mais_economica :-
    % Obtém a origem e o destino da viagem
    origem(Origem),
    destino(Destino),
    % Encontra o caminho mais econômico entre a origem e o destino
    caminho_menor_custo(Origem, Destino, Caminho, Custo, Transportes),
    nl, nl, write('O caminho mais econômico de '), write(Origem), write(' para '), write(Destino), write(' é:'), nl,
    % Inverte o caminho encontrado para exibição
    reverse(Caminho, CaminhoReverso),
    write('Caminho: '), print_caminho(CaminhoReverso), nl,
    write('Custo total: '), write(Custo), write(' Euros'), nl,
    write('Transportes utilizados: '), print_transportes(Transportes), nl.

mais_rapida :-
    % Obtém a origem e o destino da viagem
    origem(Origem),
    destino(Destino),
    % Encontra o caminho mais rápido entre a origem e o destino
    caminho_menor_tempo(Origem, Destino, Caminho, Tempo, Transportes),
    nl, nl, write('O caminho mais rápido de '), write(Origem), write(' para '), write(Destino), write(' é:'), nl,
    % Inverte o caminho encontrado para exibição
    reverse(Caminho, CaminhoReverso),
    write('Caminho: '), print_caminho(CaminhoReverso), nl,
    write('Tempo total: '), write(Tempo), write(' minutos'), nl,
    write('Transportes utilizados: '), print_transportes(Transportes), nl.

mais_economica_rapida :-
    % Obtém a origem e o destino da viagem
    origem(Origem),
    destino(Destino),
    % Encontra o caminho de menor multiplicação entre custo e tempo
    caminho_menor_multiplicacao(Origem, Destino, Caminho, Multiplicacao, Transportes),
    nl, nl, write('O caminho mais econômico e rápido de '), write(Origem), write(' para '), write(Destino), write(' é:'), nl,
    % Inverte o caminho encontrado para exibição
    reverse(Caminho, CaminhoReverso),
    write('Caminho: '), print_caminho(CaminhoReverso), nl,
    write('Total: '), write(Multiplicacao), nl,
    write('Transportes utilizados: '), print_transportes(Transportes), nl.

% Exibe o caminho
print_caminho([Origem|_]) :- write(Origem).
print_caminho([Destino, Origem|Resto]) :- print_caminho([Origem|Resto]), write(' -> '), write(Destino).

% Exibe os transportes
print_transportes([]).
print_transportes([Transporte|Resto]) :- write(' > '), write(Transporte), write(' '), print_transportes(Resto), fim.

fim :-
    retractall(origem(_)),
    retractall(destino(_)), nl, nl, nl,
    write('Obrigado pela sua preferencia! Deseja ser recomendado novamente?'), nl, nl,
    write(' "1." - Sim, por favor'), nl,
    write(' "0." - Nao. Quero sair '), nl,
    read(Z), (
        (1 == Z), nl, origem;

```

```
(0 == Z), nl, write('Terminou'), halt;  
write('Introduza uma opcao valida, por favor tente novamente!'),nl,nl, fim).
```

Anexo B

Contrato do Grupo

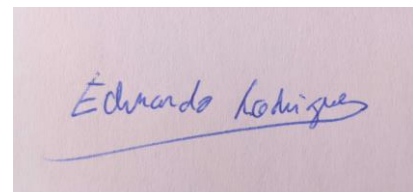
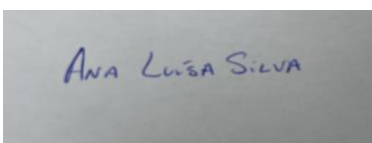
Por forma de assegurar o funcionamento correto do grupo, foram acordadas várias regras, que todos os elementos do grupo se responsabilizam por cumprir na íntegra:

- 1 - No início de cada semana faz-se uma reunião de presença obrigatória, exceto em casos devidamente justificados e aceites por todos os membros do grupo.
- 2 - Cada elemento do grupo deve realizar a tarefa que lhe foi atribuída, em caso de tarefas individuais, e deve comparecer a todas as reuniões extraordinárias para realização de tarefas conjuntas, exceto em casos devidamente justificados e aceites por todos os membros do grupo.
- 3 - A divisão de tarefas deve ser feita por forma a distribuir o mais igualmente possível o nível de dificuldade e esforço que cada tarefa necessita.
- 4 - Quando existem dúvidas na realização de uma tarefa individual, o elemento em questão é obrigado a informar os restantes membros dessa dúvida para a resolver ou se necessária marcação de reunião extraordinária.
- 5 - Cada elemento irá cumprir os prazos delineados para as suas tarefas, quando individuais, fazendo para isso uso da cláusula 4 quando necessário.
- 6 - O não cumprimento das cláusulas acima descritas, o desrespeito, falta de interesse, incumprimento de prazos ou outras falhas de carácter de igual natureza por qualquer elemento do grupo implicam a total desvalorização do seu contributo para o grupo, e consequentemente atribuição de nota 0 (zero) na sua nota final do projeto.
- 7 - Este contrato é válido e considera-se vinculativo aquando da assinatura de todos os elementos do grupo.

Ana Luísa Silva

André Macedo Barros

Carlos Rodrigues



Bibliografia

- Manual SWI-Prolog : https://www.swi-prolog.org/pldoc/doc_for?object=manual;
- Introdução à Programação Prolog, Luiz Palazzo;
- Material fornecido pelo docente nas aulas teóricas da Unidade Curricular;
- Bratko, Ivan, Prolog – PROLOG PROGRAMMING FOR ARTIFICIAL INTELLIGENCE, Longman, 2000